GROWNUPSVET

Experimento de latencia HTTP y PostgreSQL

Comparación de rutas HTTPS y del tramo Cloud Run–Render

Informe técnico de mediciones

3 de noviembre de 2026

Zona horaria de referencia: Colombia (UTC–5)

Índice

1. Diseño **2**

2. Resultados HTTP **2**

3. Tramo Cloud Run a PostgreSQL **3**

3.1. Resultados de PostgreSQL 3

4. Conclusiones **4**

5. Reproducción **4**

1 Diseño

Se midieron dos rutas HTTPS con el mismo cliente Python 3.14 en Windows, en la zona horaria de Colombia. El acceso por Cloudflare informó el punto de presencia `BOG` en el encabezado CF-Ray. La revisión de API observada en Cloud Logging fue `grownupsvet-backend-00013-jfg`, en Cloud Run Iowa (`us-central1`), con PostgreSQL externo en Render Ohio según el registro del despliegue. Las muestras HTTP se tomaron el 3 de noviembre de 2026 entre las 20:12:59 y las 20:14:31 en Colombia (UTC–5). Las marcas del CSV están en UTC.

- Control HTTP público: `GET /api/v1/hello`. Su controlador devuelve texto y no consulta PostgreSQL.
- Lectura autenticada: `GET/api/v1/appointments?from=2026-10-27&to=2026-11-10&size=10`, con la misma cuenta de ensayo y el mismo token en las dos rutas. Ambas rutas devolvieron HTTP 200 y 1511 bytes de JSON para todas las lecturas medidas. El cuerpo no se guarda.
- Por operación, cuatro pares de calentamiento y 40 pares de medición. Un par contiene una llamada vía Cloudflare Pages y otra al dominio directo de Cloud Run. Se alternó el orden de los pares, con 150 ms de pausa entre llamadas y una sola solicitud activa. El cliente intentó reutilizar una conexión HTTPS/1.1 por destino. El temporizador monotónico empieza antes del envío y termina tras recibir el cuerpo. Cualquier establecimiento de conexión realizado durante esa llamada entra en el tiempo observado.
- Cloudflare indicó `DYNAMIC` para las respuestas de la corrida principal. Se conservaron pilotos separados. Uno reveló los parámetros de fecha obligatorios mediante HTTP 400 y otro se interrumpió ante una desconexión de transporte. La corrida final contabilizó 160/160 HTTP 200 en la fase medida y no requirió reintentos. Los pilotos se excluyeron de los percentiles.
- El P95 usa rango más cercano, el valor ordenado en la posición techo(0,95 × n). La diferencia pareada es tiempo vía Cloudflare menos tiempo directo para cada par. Su P95 y la resta de los P95 por ruta son estadísticas distintas.
- `cloud-run-http.csv` contiene los eventos con método y URL de prueba hallados en la ventana de Cloud Logging. Se conserva como evidencia de la revisión. Carece de un ID de correlación con el cliente, por lo que no se usa para asignar duraciones de servidor a pares individuales. En los eventos encontrados (ambas rutas juntas, incluido calentamiento), listar citas tuvo 88 observaciones, mediana 231,82 ms y P95 244,08 ms en Cloud Run. El control tuvo 87 eventos, mediana 4,09 ms y P95 6,32 ms. Estas cifras describen la ventana de logs y no se restan de los percentiles del cliente.

2 Resultados HTTP

Tabla 1: Latencia HTTP extremo a extremo, Colombia, 3 de noviembre de 2026.

Operación	Ruta	Éxitos	Mediana (ms)	P95 (ms)
Control	Cloudflare	40/40	307,68	484,72
Control	Directa	40/40	122,50	339,88
Listar citas	Cloudflare	40/40	536,53	733,85
Listar citas	Directa	40/40	350,44	455,31

Al listar citas, Cloudflare fue más lento en 39 de 40 pares. La mediana pareada fue 192,65 ms y la resta de los P95 fue 278,55 ms. Hubo una respuesta directa de 1.759,40 ms, que se conserva en los datos. En el control, la mediana pareada fue 178,86 ms. La diferencia observada corresponde a ambas rutas completas y puede incluir tránsito, intermediación, establecimiento de conexiones y variación temporal del origen. Una sola ventana desde una red limita la generalización. El umbral histórico de 500 ms correspondía a la latencia de plataforma de ciertas operaciones de negocio. Esta lectura E2E es una comparación distinta.

3 Tramo Cloud Run a PostgreSQL

La sonda se ejecutó en un Cloud Run Job temporal, en la misma región de la API, contra la misma base externa de Render. Usó PostgreSQL cliente 18.6 (psql/libpq) y validación TLS de cadena y nombre con `verify-full`. Una conexión ya abierta ejecutó secuencialmente 65 ciclos. Cada ciclo envió `SELECT 1` en autocommit, seguido por `BEGIN READ ONLY`, otro `SELECT 1` y `COMMIT`. Se descartaron cinco ciclos de calentamiento para las estadísticas. Cada sentencia se emitió por separado y se esperó su respuesta antes de emitir la siguiente. El conteo del protocolo de la sonda es de dos consultas y dos comandos de transacción por ciclo, cuatro intercambios lógicos. Cada intercambio puede utilizar varios paquetes TCP.

La medida de psql incluye el envío y la respuesta, el trabajo de PostgreSQL y la sobrecarga del cliente. Excluye el establecimiento inicial de conexión. El `COMMIT` de la sonda cierra una transacción de solo lectura y no mide el costo de confirmar escrituras durables. El método de servicio `AppointmentService.list` contiene tres consultas JDBC de aplicación: una lectura de la cuenta, un conteo y la lectura paginada. El conteo del código se limita a ese método. La sonda no capturó los intercambios TCP/JDBC ni midió el tiempo individual de SQL o del pool dentro de las peticiones reales de la API.

3.1 Resultados de PostgreSQL

Ejecución Cloud Run Job `grownupsvet-latency-probe-200828-xqlzr`. Se obtuvieron 260 comandos válidos, de los cuales 240 entran en las estadísticas.

Tabla 2: Intercambios de PostgreSQL desde Cloud Run Iowa hacia Render Ohio.

Comando o secuencia	N	Mediana (ms)	P95 (ms)
`SELECT 1` en autocommit	60	20,11	21,59
`BEGIN READ ONLY`	60	20,02	21,44
`SELECT 1` en transacción	60	20,11	20,84
`COMMIT` de solo lectura	60	20,03	20,91
Transacción de lectura	60	60,31	62,33

La fila de transacción suma BEGIN, SELECT y COMMIT dentro de cada ciclo antes de calcular el percentil. Un intercambio simple estuvo cerca de 20 ms en esta ventana, por debajo de la referencia hipotética de 56 ms. Las esperas se acumulan cuando las llamadas son secuenciales. La prueba no permite atribuir un porcentaje de la latencia histórica de las escrituras a la red. Se exportaron los datos antes de retirar el trabajo temporal.

4 Conclusiones

En la ventana medida desde Colombia, la ruta vía Cloudflare presentó mayor latencia extremo a extremo al listar citas. El P95 fue de 733,85 ms vía Cloudflare y 455,31 ms por acceso directo a Cloud Run. La diferencia mediana dentro de cada par fue de 192,65 ms y la ruta vía Cloudflare tardó más en 39 de 40 pares. El control sin PostgreSQL también presentó una diferencia mediana de 178,86 ms entre rutas. La comparación recoge el efecto del recorrido completo, que incluye tránsito, intermediación y variación en las conexiones y en el origen.

Entre Cloud Run (Iowa) y Render (Ohio), una consulta mínima sobre una conexión TLS ya abierta tardó 20,11 ms de mediana y 21,59 ms de P95. Una transacción de solo lectura con tres comandos secuenciales tardó 60,31 ms de mediana. En esta muestra, 56 ms se aproxima más al tiempo de varios intercambios consecutivos que al de uno solo. La sonda mide el intercambio completo e incluye la ejecución de PostgreSQL y el trabajo del cliente.

Las mediciones sustentan una diferencia de latencia entre las dos rutas HTTP observadas y muestran que los intercambios secuenciales con una base remota pueden acumular espera. El aporte exacto de esa espera a los incumplimientos históricos de 500 ms al solicitar y confirmar citas sigue sin determinarse. Esas cifras corresponden a otras operaciones y se midieron del lado de Cloud Run, mientras que la comparación entre rutas mide desde el cliente. La sonda fue de solo lectura y no midió la adquisición de conexiones, los bloqueos ni la confirmación de escrituras dentro de la API.

Para cuantificar ese aporte, una repetición con datos ficticios debería correlacionar cada petición de escritura con los tiempos del pool, las consultas JDBC, la ejecución y las esperas en PostgreSQL y el `COMMIT`. Repetir la comparación HTTP en otras ventanas y redes ayudaría a determinar cuánto se mantiene la diferencia observada.

5 Reproducción

`medir-http.py` recibe por entrada estándar un JSON con las credenciales de ensayo y conserva solo metadatos HTTP. `medir-postgres.sh` es el cuerpo del trabajo; requiere los parámetros de conexión como variables PG y la contraseña desde la referencia existente de Secret Manager. `extraer-postgres.py` valida que existan las 260 mediciones en los eventos de una única ejecución y exporta el CSV. `resumir.py` recalcula las tablas a partir de los CSV. Usar una carpeta nueva para conservar la muestra original.

- Definición de latencia del servidor HTTP
- Regiones de Cloud Run
- Cliente psql y temporización
- Cadena pública de Let’s Encrypt